

Vibe Project Management App Plan (Using Shared Hostinger Server)

Executive Summary

Vibe will be an agile project-management app modeled on Zoho Sprints. Core features include backlog management, sprint planning, issue tracking on boards (Scrum/Kanban), customizable workflows, custom fields, reporting, permissions, and notifications. We will build Vibe using Node.js on a shared Hostinger server (157.173.208.123) that supports Node 18–24 and frameworks like Express, NestJS, React, Vue, etc. The database will likely be MySQL or MongoDB hosted on the same plan or externally. The frontend will be a modern JS SPA (React, Vue, or Svelte) with real-time updates via WebSockets or SSE. Key deliverables: a Minimum Viable Product (MVP) with user/projects auth, issue backlog, sprint board, and CI/CD deployment. Later enhancements include advanced reports, offline support, and more. Development will follow Agile sprints, with milestones for planning, MVP release, beta testing, and v1.0 launch.

This report covers mapping Zoho Sprints features to Vibe's design, data models (with Mermaid diagrams), API approaches (REST/GraphQL), real-time strategies, frontend architecture options, state management, conflict resolution, offline support, auth/roles, testing, and CI/CD deployment — all tailored to the shared Hostinger environment and its constraints.

Core Features (Zoho Sprints Equivalent)

Zoho Sprints features guide Vibe's functionality. We identify the following **core features** for MVP, based on Zoho's documentation:

- **Backlog Management:** All unplanned work items are visible in a backlog view ¹. In Vibe, users can create/edit issues with fields like title, description, type (bug/story/task), priority, estimate, assignee, etc. The backlog list allows filtering/sorting by these fields.
- **Sprints:** Users can define Sprints (name, start/end dates, goals) and move backlog items into them. Zoho supports planning multiple releases; in Vibe MVP, a single-board sprint per project suffices.
- **Task Board (Scrum/Kanban):** The active sprint is visualized on a board. Zoho's Scrum board shows work items by status ². Vibe's board will have default columns (e.g., To Do, In Progress, Done) and allow drag/drop moves that update status instantly. The board should reflect any custom statuses (next point).
- **Workflows/Statuses:** Projects define workflow statuses. Zoho lets you "create custom workflow statuses" ². Vibe will allow admins to configure column names and workflow states (at least a simple 3-state workflow to start).

- **Custom Fields & Layouts:** Zoho allows adding custom fields per project ³. Vibe will support additional issue fields (e.g., client, severity) via a schema that can be extended.
- **Reports & Analytics:** Basic agile reports (sprint burndown, velocity chart) are in Zoho ⁴. For MVP, include a sprint burndown graph using charting libraries. Enhance later with velocity, cumulative flow, etc.
- **Permissions & Roles:** Zoho has Profiles/Roles for access control ⁵. Vibe will implement role-based access (e.g., Admin, Developer, Viewer) on projects. Admins can invite/assign users to projects, defining who can edit or just view.
- **Notifications:** Automated notifications on key events (issue created, status changed). Zoho provides email templates for updates ⁶. Vibe will send emails or in-app alerts (via WebSocket events) to relevant users when issues or sprints are updated.
- **Comments/Collaboration:** Issues will have threaded comments (not explicitly in Zoho features, but standard).
- **Integrations/Plugins (Future):** Not in MVP per scope.

MVP vs Later: For MVP, focus on backlog view, sprint creation, board drag-drop, basic issue fields, auth/roles, and simple reports. Later, add epics, dependencies, time tracking, advanced reports, and offline support.

Data Model (Mermaid ER Diagram)

A high-level ER diagram for Vibe's backend is shown below (core entities only). This guides our database schema design (relational or document).

```
erDiagram
    USER ||--o{ PROJECT : owns
    USER ||--o{ ISSUE : created
    USER ||--o{ COMMENT : writes
    PROJECT ||--o{ SPRINT : has
    PROJECT ||--o{ ISSUE : contains
    SPRINT ||--o{ ISSUE : includes
    ISSUE ||--o{ COMMENT : has
    ISSUE ||--o{ ISSUE : blocks
```

- **User:** System user (with roles). A user owns 0+ Projects (as admin). Users can create Issues and Comments.
- **Project:** Container for sprints and backlog. Has an owner and team members.
- **Sprint:** Has dates (start/end) and goals, belongs to one Project, contains many Issues.
- **Issue (Work Item):** Belongs to a Project and optionally a Sprint. Has fields (title, type, status, priority, assignee, etc.). Issues can block other issues (self-link).

- **Comment:** Linked to one Issue, created by one User.

This model can be implemented in SQL (PostgreSQL/MySQL) or NoSQL (MongoDB). For relational DB, tables (users, projects, sprints, issues, comments) with foreign keys. For NoSQL, documents with references or embedded sub-documents.

API Design (REST vs. GraphQL)

We will expose a JSON API for frontend. Two common styles: REST or GraphQL. Both can work given Hostinger supports Express/NestJS (for REST) and many GraphQL libraries.

REST (Express/Nest): Traditional approach with endpoints like:

- POST /api/users/register (signup)
- POST /api/users/login (JWT auth)
- GET /api/projects (list), POST /api/projects
- GET /api/projects/:id/issues (backlog list), POST /api/projects/:id/issues
- GET /api/projects/:id/sprints, POST /api/projects/:id/sprints
- PATCH /api/projects/:pid/issues/:iid (update status)
- etc.

This style is simple to cache and fits well with Express.

GraphQL (Apollo Server/Nest): Single endpoint /graphql. Clients query exactly needed fields. Example schema snippet:

```
type Issue { id: ID!, title: String!, status: String!, assignee: User, ... }
type Query { projectIssues(projectId: ID!): [Issue!]! }
type Mutation { createIssue(input: CreateIssueInput!): Issue }
```

GraphQL lets the frontend fetch complex, nested data in one call and supports real-time **Subscriptions** (over WebSockets). AWS notes “GraphQL uses a server-side schema... REST APIs do not require a schema” ⁷. For real-time, GraphQL Subscriptions (with issueUpdated events) can broadcast via WS automatically ⁸. However, GraphQL adds complexity (schema, resolvers) and requires additional libraries (Apollo or Relay).

Comparison Table – API Options:

API Style	Endpoint Pattern	Real-time Support	Caching	Complexity
REST	Multiple URLs (e.g. / issues, /sprints)	Use WebSocket/ SSE on side	HTTP caching per route	Straightforward, lower learning curve
GraphQL	Single /graphql with queries/ mutations	Built-in Subscriptions (WS) ⁸	Difficult (caching by query)	Powerful but higher initial effort

Given our small app and shared-host constraints, a REST API (Express or Nest) plus a separate WebSocket server (Socket.IO) may be simpler to deploy. Alternatively, we can use Apollo GraphQL with subscriptions if we want tight integration.

Sample Node.js Handlers

REST Endpoint (Express): Create a new issue in backlog.

```
app.post('/api/projects/:projectId/issues', authenticateJWT, async (req, res) => {
  const { title, description, type, priority, assigneeId } = req.body;
  const { projectId } = req.params;
  // Save to DB (validating permissions, project exists, etc.)
  const issue = await Issue.create({ projectId, title, description, type,
    priority, assigneeId, status: 'To Do', createdBy: req.user.id });
  res.status(201).json(issue);
});
```

WebSocket Handler (Socket.IO): Broadcast issue updates.

```
io.on('connection', socket => {
  socket.on('joinProject', projectId => socket.join(`project-${projectId}`));
  socket.on('updateIssue', (issue) => {
    // Assume issue has {id, projectId, ...}
    io.to(`project-${issue.projectId}`).emit('issueUpdated', issue);
  });
});
```

Developer AI Prompt Example:

"Generate a Node.js Express route for `PATCH /api/issues/:id` that updates an issue's status. Include JWT auth check and optimistic concurrency (using a version field or `updatedAt` check)."

This prompt can be fed to an AI coding assistant (like OpenAI Codex/OpenCode) to quickly scaffold the logic.

Real-Time Update Strategy

To avoid full page refreshes, Vibe's UI must update in real time:

- **WebSockets (Socket.IO):** Enables full-duplex communication. Clients connect and listen for events. When one client (user) moves a task or edits it, the server broadcasts the change to others. E.g.,

sending an `issueUpdated` event. MDN notes WebSocket “opens a two-way interactive communication session” ⁹. We will use Socket.IO (Node) and a small room-per-project pattern.

- **Server-Sent Events (SSE):** One-way (server→client). Good for notifications or live feeds (e.g. comment feed). The server pushes events; clients use `EventSource`. We should implement one-way notifications via SSE where appropriate (e.g. someone creates a comment, everyone’s UI can listen via SSE and append it).
- **Polling:** Simplest fallback (e.g. fetch every 5–10 sec), but inefficient. With unlimited bandwidth on our plan, polling is possible but wastes resources.
- **Optimistic UI:** On the frontend, apply updates before server confirmation to improve perceived speed. For example, when a user drags a card to “Done”, we immediately move it in the UI and send the request. If the server later fails, we roll back. This pattern (used by React Query, Apollo) greatly enhances UX ¹⁰.

Real-Time Options Comparison:

Method	Direction	Use Case	Notes
WebSockets	Bi-directional	Task board updates, chat, comments	Low-latency, event-driven; supported by Hostinger’s Node env. MDN: “two-way interactive communication” ⁹ .
SSE	Server→Client only	Notifications, live feeds	Simpler than WS. MDN: server can push data to page ¹¹ . Client auto-reconnects on drop.
Polling	Client→Server pull	Fallback, simple updates	Higher latency and server load; not ideal if true real-time needed.
WebRTC	Peer-to-peer	Audio/Video (not needed here)	Out of scope for this app (no P2P media).

For Vibe’s backlog and board updates, **WebSockets** is the most robust. **Optimistic UI** logic (for both WS and HTTP updates) will make the experience seamless.

Frontend Architecture & State Management

We will build a Single Page App. Options for frontend framework (supported by server) include React, Vue, Svelte, Angular, etc. Key considerations:

- **React:** Popular, large ecosystem. Use libraries like React Router for navigation and Redux Toolkit or Recoil for state. React Query (or SWR) can manage server state and optimistic updates. React works well with JSX and has many UI component libraries (Material-UI, AntD).
- **Vue.js:** Excellent templates and reactivity. Use Vuex (or Pinia) for state. Very approachable. Integrates well with Vue Router.

- **Svelte/SvelteKit:** Compiles to minimal JS, very fast. SvelteKit can handle routing. State is built-in or via stores. A good choice if performance is crucial, though ecosystem smaller.
- **Angular:** Full framework with TypeScript. Has built-in state management (NgRx) and RxJS. Steeper learning and heavier bundle; likely overkill for a lean app.
- **Others (Next.js, Nuxt, etc):** These are meta-frameworks (SSG/SSR) built on React/Vue. For simplicity, we'll stick to a SPA (CSR) setup.

State Management: Must handle both client state (UI) and server data. Options:

- **Redux Toolkit (React):** Central store, good for complex global state. Can implement optimistic updates manually or with RTK Query.
- **Vuex/Pinia (Vue):** Similar store pattern for Vue.
- **Context/Reducer (React):** Lighter-weight if state is simple.
- **Apollo Client (GraphQL):** If we choose GraphQL, its normalized cache can manage state and optimistic updates built-in.
- **SWR/React Query:** Fetch/Cache library that supports mutations with optimistic updates, works with REST or GraphQL.

Compare Frontend Frameworks:

Framework	Language	Data Binding	State Mgmt	Ecosystem/Notes
React	JS/ TypeScript	One-way (props/ hooks)	Redux, Context, React Query	Huge ecosystem; works well with Socket.IO and hooks for WS.
Vue 3	JS/ TypeScript	Two-way (v- model)	Vuex/Pinia, Composition API	Developer-friendly; built-in reactivity; good router.
Svelte	JS/ TypeScript	Reactive assignments	Svelte stores	Very fast, small bundle; rising popularity.
Angular	TypeScript	Two-way (ngModel)	NgRx (Redux style)	Full-featured framework; more overhead.

Given the shared server's Node support, all are viable. **React or Vue** are recommended for their balance of simplicity and ecosystem. In either case, we will architect components (BacklogList, SprintBoard, IssueDetail, etc.) that consume the API, connect to WebSocket events, and update state optimistically.

Conflict Resolution and Offline Support

Concurrency Control: If two users edit the same issue simultaneously, we need to avoid lost updates. Strategies:

- *Optimistic Locking:* Keep a version or `updatedAt` timestamp on each issue. When updating, include the version; if the DB version has changed, reject the update (HTTP 409) and prompt a refresh. AWS recommends "optimistic locking... detects conflicts at write time using conditional writes" ¹².
- *Last-Write-Wins:* Simply overwrite (may lose data). Risky for concurrent edits.
- *Field-Level Merge:* Merge non-conflicting fields and mark conflicts (complicated to implement).
- *Operational Transforms/CRDTs:* Overkill for tasks; meant for real-time collaborative text editing ¹³.

For MVP, use optimistic version checks and simple retry.

Offline Support (Progressive Web App): The shared hosting environment can serve a PWA. We will use a Service Worker to cache the app shell and API responses ¹⁴. This allows basic offline viewing of last-loaded data. Background Sync can queue updates (e.g. issue creation) while offline, retrying when online ¹⁵. Offline mode ensures Vibe remains usable even if the network disconnects temporarily.

Authentication & Authorization

We will implement secure login and access control:

- **Authentication (AuthN):** Use JWTs (JSON Web Tokens) for stateless auth. On successful login/signup, server issues a JWT (signed with a secret) ¹⁶. The client stores the token (e.g. in an HTTP-only cookie or memory) and sends it in `Authorization: Bearer <token>` with API requests. JWTs contain user ID and role, and have a short expiration for security.

Example (Node.js):

```
const token = jwt.sign({ userId: user.id, role: user.role },
  process.env.JWT_SECRET, { expiresIn: '8h' });
```

- **Authorization (AuthZ):** Implement Role-Based Access Control (RBAC). As one guide explains, “RBAC allows you to define roles (e.g. ‘admin’, ‘editor’, ‘viewer’) and assign permissions to each role” ¹⁷. In Vibe, we’ll check roles in middleware: only Admins can create projects, project members can create issues, etc. Teams are per-project, so users need to be invited.

Security best practices: always use HTTPS, hash passwords with bcrypt, validate and sanitize inputs, and manage secrets (rotate JWT secret periodically) ¹⁸. Avoid storing JWTs in localStorage to prevent XSS; prefer httpOnly cookies if possible.

Testing Strategy

Vibe must be well-tested:

- **Unit Tests:** Cover utility functions, data models, and core business logic. Use Jest or Mocha (for Node) and React Testing Library or Vue Test Utils for components. Example: test that creating an issue with invalid data returns an error.
- **Integration Tests:** Test API endpoints and database interactions. Use Supertest or Jest on the Express/Nest server. For example, create a test database, then send HTTP requests to `/api/sprints` and verify the response.
- **End-to-End (E2E) Tests:** Simulate user workflows in a browser. Tools: **Cypress** or **Playwright**. Write tests like: user logs in, creates a sprint, drags an issue on the board, and verifies UI changes. E2E ensures the entire stack works together.

- **Test Environments:** Use a separate staging environment on Hostinger (or local Docker) to run tests. If the shared server plan allows multiple sites, we could deploy a staging instance to a subdomain.
- **Continuous Testing:** Integrate tests into CI (see next section) so every push triggers automated tests.

CI/CD and Deployment (Shared Hostinger Server)

We will set up Continuous Integration/Continuous Deployment as follows:

- **Source Control:** Code in GitHub or GitLab. Enforce code reviews.
- **CI Pipelines:** Use GitHub Actions (free with GitHub) or GitLab CI. The pipeline runs on each commit: `npm test` (unit/integration), lint checks, and build (transpile JS).
- **Deployment:** Our Hostinger plan is shared hosting (server1537, USA). It supports Node.js (versions 18.x–24.x) and popular frameworks (Express, Nest, React, Vue, etc.) as listed. We have 1536 MB RAM and 2 CPU cores, so we should optimize memory usage (e.g. avoid huge in-memory caches). We will deploy by:
 - **Building** the app (e.g. `npm run build` for frontend + backend transpilation).
 - **Uploading files** to `public_html` on the server. Hostinger allows Git deployment or FTP. Here we have FTP details:
 - **FTP Host:** `resource.aiitsolutions.com`
 - **FTP User:** `u298324660.resource.aiitsolutions.com`
 - Files go in `public_html`. We can push via command line (`lftp`), or via GitHub Actions using FTP plugins.
 - **Process Manager:** Use PM2 (or Hostinger's Node app manager) to run the Node server. PM2 can be installed on the server if SSH is allowed, or use Hostinger's dashboard.
 - **Environment:** Since Node is supported, ensure we pick a Node version (e.g. 20.x) that matches dev environment.
 - **DNS Configuration:** The domain is set to Hostinger nameservers (ns1.dns-parking.com, ns2). Point `resource.aiitsolutions.com` and `www.resource.aiitsolutions.com` to the app. Hostinger will manage SSL certificates automatically for these domains.
- **CI/CD Example (GitHub Actions):** A workflow that on push to `main`:
 - Installs Node 20 (`actions/setup-node`).
 - Runs `npm ci && npm test`.
 - Builds the app (`npm run build`).
 - Uses an FTP Action to upload `./build` to Hostinger's `public_html`.

- **Monitoring:** Use lightweight monitoring. Hostinger may offer basic stats; for deeper insight, integrate a service like UptimeRobot (ping the API) and Sentry (capture frontend/backend errors). Since we have limited memory, avoid heavy on-server monitoring agents; log minimal metrics.
- **Backups:** The plan includes 200 GB disk and daily backups in Boston. Still, we should periodically export the database (or rely on Hostinger's backup if available). Store backups externally if possible.

Deployment Table – Host vs Requirements:

Aspect	Hostinger Shared Plan	Considerations
RAM	1536 MB	Optimize Node memory usage, no heavy services.
CPU	2 cores	Can handle moderate traffic; scale vertically if needed.
Disk	200 GB (SSD)	Ample for code and DB.
Node Support	v18.x,20.x,22.x,24.x	Use latest LTS (e.g., Node 20).
Frameworks	Express, Nest, React, Vue...	All needed tools are supported.
Processes	Max 120 processes	One PM2 process per core is fine.
Bandwidth	Unlimited	No rate-limit concerns.
CI/CD	No built-in CI, need FTP/SSH	Use external CI tools + FTP deploy.
Monitoring	Limited	Use external tools (e.g. UptimeRobot).
SSL	Hostinger auto-SSL	Ensure certificates for both URLs.

Scalability and Security

With this shared environment, horizontal scaling is limited. We rely on the 2 CPU cores and memory. To handle growth:

- **Vertical Scaling:** If needed, upgrade Hostinger plan (more RAM/cores).
- **Stateless Services:** Ensure Node app is stateless (JWT for auth). Use Redis or DB for shared state if multiple instances.
- **Caching:** Use in-memory cache sparingly. With 1536 MB RAM, prefer Redis or file caching (if available on host) for session store or frequent queries.
- **Database:** For MySQL on Hostinger or MongoDB Atlas, ensure indexes on frequently accessed fields (projectId, status).
- **Security:** Follow OWASP. The shared server uses Hostinger nameservers, so ensure DNS is locked down. Use HTTPS. Sanitize all inputs (SQL and NoSQL injection prevention). Keep dependencies up-to-date. Use Content Security Policy and other HTTP headers.

Development Roadmap and Milestones

1. Phase 1 – Planning & Setup (2–3 weeks):

2. Finalize requirements and architecture (DB choice, API style).
3. Provision Hostinger Node environment (set up Git/FTP, Node version, SSL).
4. Set up version control and CI pipeline.

5. Phase 2 – Core MVP (8–10 weeks):

6. **Backend:** Implement auth (JWT), user/project APIs, issue CRUD, sprint endpoints. Add WebSocket server.
7. **Frontend:** Build SPA (React/Vue) with pages for login, project dashboard, backlog list, and sprint board.
8. **Features:** Backlog creation, sprint planning, board drag/drop, real-time updates via WebSocket.
9. **Deploy:** Initial deploy to Hostinger staging subdomain for testing.

Milestone: Demo of backlog and board functionality with real-time updates on staging.

1. Phase 3 – Enhancements (4–6 weeks):

2. **UI/UX Polish:** Add custom fields UI, filters, user management (invite users to projects).
3. **Notifications:** Implement email and on-screen alerts for key events.
4. **Testing:** Expand unit/integration/E2E tests.
5. **Performance:** Optimize DB queries, add caching (e.g. Redis if possible).

Milestone: Feature-complete beta version, stable for production use.

1. Phase 4 – Production Launch (2 weeks):

2. Migrate staging to production (`resource.aiitsolutions.com`).
3. Conduct load/stress testing (simulate concurrent users).
4. Final security audit and fixes.
5. Release v1.0.

6. Phase 5 – Post-Launch (Ongoing):

7. Gather user feedback and iterate (bug fixes, UX improvements).
8. Implement next-phase features (advanced reports, offline support).
9. Monitor usage and plan for scaling or integration as needed.

Risk Mitigation: Regular demos will catch requirement gaps early. Using the known Hostinger tech stack reduces surprises. Automated tests and CI will prevent regressions. Security scans (npm audit) will guard against vulnerabilities.

Citations

- Zoho Sprints features (backlog, boards, workflows) ¹ ² .
- Zoho custom fields & roles ³ ⁵ .
- Optimistic UI pattern ¹⁰ .
- AWS on optimistic locking ¹² .
- StackOverflow on concurrency/locking ¹³ .

- MDN SSE description ¹¹ .
- MDN WebSocket API overview ⁹ .
- MDN JWT intro (RFC7519) ¹⁶ (for understanding JWT use).

These sources underpin our design choices and ensure we follow best practices in building Vibe on the given hosting platform.

¹ ² ³ ⁴ ⁵ ⁶ Agile Project Management Software Features | Zoho Sprints

<https://www.zoho.com/sprints/features.html>

⁷ ⁸ GraphQL vs REST API - Difference Between API Design Architectures - AWS

<https://aws.amazon.com/compare/the-difference-between-graphql-and-rest/>

⁹ WebSocket API (WebSockets) - Web APIs | MDN

https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

¹⁰ Understanding optimistic UI and React's useOptimistic Hook - LogRocket Blog

<https://blog.logrocket.com/understanding-optimistic-ui-react-useoptimistic-hook/>

¹¹ Server-sent events - Web APIs | MDN

https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events

¹² Best practices for handling concurrent updates in DynamoDB - Amazon DynamoDB

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/BestPractices_ImplementingVersionControl.html

¹³ concurrency - REST API: How to prevent races when doing read - update - write cycle on server side - Stack Overflow

<https://stackoverflow.com/questions/79282527/rest-api-how-to-prevent-races-when-doing-read-update-write-cycle-on-server>

¹⁴ ¹⁵ Offline and background operation - Progressive web apps | MDN

https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Offline_and_background_operation

¹⁶ JSON Web Token Introduction - jwt.io

<https://www.jwt.io/introduction>

¹⁷ ¹⁸ Node.js Authentication: JWT, Refresh Tokens, OAuth & Roles

<https://wontonee.com/node-js-authentication-jwt-refresh-tokens-oauth-roles/>